LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 7



IMMANUEL RADITYA DEO PRATAMA PAMBUDHI
103132430001
S1SD-05-A

PROGRAM STUDI SAINS DATA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

Dasar Teori

1. Kelas Abstrak (Abstract Class)

Kelas abstrak adalah kelas yang dideklarasikan dengan kata kunci abstract dan tidak dapat diinstansiasi secara langsung menjadi objek. Kelas ini berfungsi sebagai cetak biru bagi kelas-kelas turunannya. Kelas abstrak dapat memuat atribut biasa, konstruktor, method konkret, serta abstract method yang mewajibkan seluruh kelas turunan konkretnya untuk melakukan override dan memberikan implementasi nyata.

2. Antarmuka (Interface)

Interface adalah tipe referensi dalam Java yang berisi konstanta dan deklarasi method abstrak yang bertindak sebagai kontrak perilaku formal. Berbeda dengan kelas yang hanya mendukung single inheritance, Java mengizinkan multiple inheritance melalui interface, di mana sebuah kelas dapat mengimplementasikan lebih dari satu interface sekaligus menggunakan kata kunci implements. Seluruh method yang dideklarasikan dalam interface secara default bersifat public dan abstract.

3. Default dan Static Method

Interface diperkaya dengan kemampuan menyediakan implementasi method tanpa merusak kelas-kelas yang telah mengimplementasikannya:

- Default Method: Method berkata kunci default di dalam interface yang memiliki bodi implementasi standar. Method ini otomatis diwarisi oleh seluruh kelas pengimplementasi tanpa memaksa kelas tersebut untuk melakukan override, namun tetap dapat di-override bila diperlukan.
- Static Method: Method utilitas berkata kunci static di dalam interface yang memiliki bodi dan dipanggil langsung melalui nama interface, misal: InterfaceName.methodName()) tanpa membutuhkan instansiasi objek.

4. Polimorfisme Heterogen

Polimorfisme memungkinkan objek dari kelas-kelas yang berbeda dikelola secara seragam melalui tipe referensi interface bersama.

Guided

Guided 1: Kelas Abstrak (Employee.java, SalariedEmployee.java, dan Main.java)

Employee.java:

```
package guided.gd1;

public abstract class Employee {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    // every employee must implement earnings method but different employees
will have different implementations
    public abstract double earnings();
}
```

SalariedEmployee.java:

```
package guided.gd1;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }
```

```
    @Override
    public double earnings() {
        return weeklySalary;
    }

}
```

Main.java:

```
package guided.gd1;

public class Main {

    public static void main(String[] args) {
        SalariedEmployee salaried = new SalariedEmployee("Deo", 1000000);
        System.out.println("Nama: " + salaried.getName());
        System.out.println("Gaji: Rp" + salaried.earnings());
    }
}
```

Output :

```
C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-07-abstract-dan-interface\project\src\main\java>java guided.gd1.Main
Nama: Deo
Gaji: Rp1000000.0

C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-07-abstract-dan-interface\project\src\main\java>_
```

Guided 2: Interface Dasar (Payable.java, Invoice.java, Employee.java, SalariedEmployee.java, dan Main.java)

Payable.java:

```
package guided.gd2;

public interface Payable {
```

```
    double getPaymentAmount();
}
```

Invoice.java:

```
package guided.gd2;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
    }

    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }

}
```

Employee.java:

```
package guided.gd2;

public abstract class Employee implements Payable {
    private String name;
```

```
    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    // every employee must implement earnings method but different employees
will have different implementations
    public abstract double earnings();

    // if there's a due payment, the amount will be the same as the earnings of
the employee
    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

SalariedEmployee.java:

```
package guided.gd2;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
```

```
}
```

Main.java:

```
package guided.gd2;

public class Main {
    public static void main(String[] args) {
        Payable[] items = new Payable[2];

        items[0] = new Invoice("Monitor LED", 2, 150000);
        items[1] = new SalariedEmployee("Deo", 5000000);

        for (Payable item: items) {
            System.out.println("Jumlah yang perlu dibayar: " +
item.getPaymentAmount());
        }
    }
}
```

Output :

```
C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-07-abstract-dan-interface\project\src\main\java>java guided.gd2.Main
Jumlah yang perlu dibayar: 300000.0
Jumlah yang perlu dibayar: 5000000.0

C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-07-abstract-dan-interface\project\src\main\java>_
```

Guided 3: Default dan Static Method pada Interface (Payable.java, Invoice.java, Employee.java, SalariedEmployee.java, dan Main.java)

Payable.java:

```
package guided.gd3;
```

```java
public interface Payable {
    double getPaymentAmount();

    // fungsi baru
    default void printReceipt() {
        System.out.println("Struk Resi - Total: Rp" + getPaymentAmount());
    }

    // fungsi utilitas
    static void logTransaction(String message) {
        System.out.println("[AUDIT LOG] " + message);
    }
}
```

Invoice.java:

```java
package guided.gd3;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
    }

    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
```

```
    }

}
```

Employee.java:

```
package guided.gd3;

public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    // every employee must implement earnings method but different employees
will have different implementations
    public abstract double earnings();

    // if there's a due payment, the amount will be the same as the earnings of
the employee
    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

SalariedEmployee.java:

```
package guided.gd3;

public class SalariedEmployee extends Employee {
```

```
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }

}
```

Main.java:

```
package guided.gd3;

public class Main {
    public static void main(String[] args) {
        Payable.logTransaction("Sistem pembayaran diinisialisasi.");

        Payable[] items = new Payable[2];
        items[0] = new Invoice("RAM DDR5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Deo", 5000000);

        for (Payable item: items) {
            item.printReceipt();
        }

        Payable.logTransaction("Transaksi selesai diproses.");

    }
}
```

Output :

```
C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-07-abstract-dan-interface\project\src\main\java>java guided.gd3.Main
[AUDIT LOG] Sistem pembayaran diinisialisasi.
Struk Resi - Total: Rp4000000.0
Struk Resi - Total: Rp5000000.0
[AUDIT LOG] Transaksi selesai diproses.

C:\Users\deoim\FIle_Kuliah_Tugas\PBO-Praktikum\modul-07-abstract-dan-interface\project\src\main\java>_
```

Unguided

Soal Unguided 1

Sebuah bioskop memerlukan sistem kasir terpadu yang mampu memproses pembayaran dalam satu keranjang belanja. Di kasir, penonton tidak hanya membeli Tiket Nonton, tetapi juga bisa membeli makanan/minuman. Secara konsep sistem, Tiket Bioskop dan Paket Snack berasal dari silsilah keluarga yang sepenuhnya berbeda. Tiket film murni bersifat abstrak, sedangkan tiket dan makanan disatukan melalui antarmuka ItemTransaksi, dihitung total tagihannya, serta dicetak dalam satu struk otomatis memanfaatkan default dan static method.

Jawaban :

ItemTransaksi.java:

```java
package unguided;

public interface ItemTransaksi {
    double getNominalBayar();
    String getNamaTransaksi();

    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi() + " | Total: Rp" +
getNominalBayar());
    }

    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
```

```
    }
}
```

TiketBioskop.java:

```
package unguided;

public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar, int jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return this.judulFilm;
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public int getJumlahKursi() {
        return this.jumlahKursi;
    }

    public abstract double hitungTotalBayar();

    public abstract String getNamaTransaksi();

    @Override
    public double getNominalBayar() {
```

```
            return hitungTotalBayar();
        }
    }
```

TiketReguler.java:

```
package unguided;

public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar, int jumlahKursi,
int nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    public int getNomorStudio() {
        return this.nomorStudio;
    }

    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + this.nomorStudio + ") - " +
getJudulFilm() + " (" + getJumlahKursi() + " x Rp" + getHargaDasar() + ")";
    }
}
```

TiketVIP.java:

```
package unguided;
```

```
public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;

    public TiketVIP(String judulFilm, double hargaDasar, int jumlahKursi,
double biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    public double getBiayaFasilitasVIP() {
        return this.biayaFasilitasVIP;
    }

    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + this.biayaFasilitasVIP) * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket VIP Premiere - " + getJudulFilm() + " (" +
getJumlahKursi() + " x Rp" + (getHargaDasar() + this.biayaFasilitasVIP) + ")";
    }
}
```

PaketSnack.java:

```
package unguided;

public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi, double hargaPerPorsi)
```

```
{
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }

    public String getNamaPaket() {
        return this.namaPaket;
    }

    public int getJumlahPorsi() {
        return this.jumlahPorsi;
    }

    public double getHargaPerPorsi() {
        return this.hargaPerPorsi;
    }

    @Override
    public double getNominalBayar() {
        return this.jumlahPorsi * this.hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + this.namaPaket + " (" + this.jumlahPorsi + " x
Rp" + this.hargaPerPorsi + ")";
    }
}
```

Main.java:

```
package unguided;

public class Main {
    public static void main(String[] args) {
```

```
        ItemTransaksi.catatLogKasir("Membuka transaksi baru untuk Pelanggan
#042.");

        TiketReguler tiketReguler = new TiketReguler("Interstellar", 40000, 2,
3);
        TiketVIP tiketVIP = new TiketVIP("Dune: Part Two", 60000, 2, 25000);
        PaketSnack paketSnack = new PaketSnack("Combo Popcorn Caramel + 2
Soda", 1, 55000);

        ItemTransaksi[] daftarTransaksi = new ItemTransaksi[3];
        daftarTransaksi[0] = tiketReguler;
        daftarTransaksi[1] = tiketVIP;
        daftarTransaksi[2] = paketSnack;

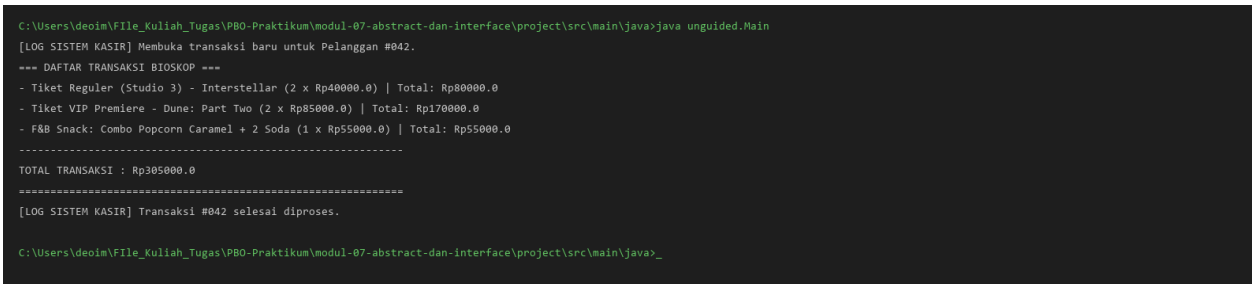

        System.out.println("=== DAFTAR TRANSAKSI BIOSKOP ===");
        double totalTransaksi = 0.0;
        for (ItemTransaksi item : daftarTransaksi) {
            item.cetakRincianItem();
            totalTransaksi += item.getNominalBayar();
        }

        System.out.println("-------------------------------------------------
----------");
        System.out.println("TOTAL TRANSAKSI : Rp" + totalTransaksi);

System.out.println("==============================================================
==");

        ItemTransaksi.catatLogKasir("Transaksi #042 selesai diproses.");
    }
}
```

Output :



Penjelasan :

Pada tugas unguided untuk Modul 7 ini di mana saya membuat sistem Cinema POS yang mengintegrasikan kelas abstrak, kontrak antarmuka, dan polimorfisme heterogen. Implementasi pertama dari kelas abstrak adalah TiketBioskop yang mencegah objek tiket untuk dibuat tanpa memilih tipe studio. Kelas ini berisi atribut privat judulFilm, hargaDasar, dan jumlahKursi, beserta deklarasi sebuah metode abstrak yang akan diimplementasikan oleh kelas anaknya seperti TiketReguler atau TiketVIP.

Kontrak interface & polimorfisme heterogen terjadi pada interface ItemTransaksi yang berfungsi sebagai kontrak umum untuk entitas yang dapat ditransaksikan di kasir. Sementara itu, misalnya menggunakan implementasi ItemTransaksi dengan memetakan metode getNominalBayar() ke metode internal hitungTotalBayar(). Di sisi lain, kelas PaketSnack yang tidak memiliki hubungan pewarisan dengan tiket juga secara independen mengimplementasikan ItemTransaksi, sehingga dua kategori barang yang berbeda dengan hierarki masing-masing dapat ditampung dalam satu array heterogen, yaitu ItemTransaksi [].

Metode default dan static diterapkan secara langsung dengan menggunakan metode default cetakRincianItem() pada ItemTransaksi agar format cetakan baris struk penerimaan menjadi otomatis distandardisasi … [14] Artinya, metode static catatLogKasir() dibuat sebagai utilitas untuk kebutuhan bisnis untuk mencatat catatan transaksi kasir melalui proses pencetakan tanpa memerlukan instansiasi objek.

Kesimpulan

Berdasarkan praktikum modul 7 yang telah diselesaikan, beberapa kesimpulan penting yang aku peroleh adalah:

1. Abstract Class mampu memberikan struktur dasar dan memaksakan standarisasi method pada subclass tanpa memungkinkan pembuatan objek yang belum terdefinisi secara konkret.
2. Interface berfungsi sebagai kontrak perilaku yang menjembatani entitas-entitas dari silsilah hierarki yang berbeda agar dapat berinteraksi secara seragam di dalam sistem.
3. Default method mempermudah penambahan fungsionalitas turunan tanpa merusak kelas yang sudah ada, sedangkan static method menyediakan fungsi helper yang terpusat.
4. Polimorfisme Heterogen: Memungkinkan array tunggal bertipe interface menampung beragam objek lintas hierarki seperti tiket bioskop dan makanan ringan, menyederhanakan pemrosesan transaksi belanja.